

A Novel Approach to SBOM Extraction from HTTP Messages in Identity Management Security Testing

Andrea Bisegna¹[0000–0002–8055–5262], Laura Cristiano¹[0000–0003–0895–3466],
Pietro De Matteis²[0009–0006–7684–7189], Eleonora
Marchesini¹[0009–0000–5508–633X], Luca Piras¹[0000–0002–7530–4119], and Silvio
Ranise^{1,3}[0000–0001–7269–9285]

¹ Center for Cybersecurity, Fondazione Bruno Kessler, Trento, Italy
{a.bisegna,l.cristiano,emarchesini,l.piras, ranise}@fbk.eu

² Co-Innovation Lab, FBK -Dedagroup SpA, Trento, Italy
pietro.dematteis@dedagroup.it

³ Department of Mathematics, University of Trento, Italy

Abstract. Modern Identity Management (IdM) deployments are distributed across multiple entities. In federated architectures, they involve Identity Providers and Relying Parties; in Self-Sovereign Identity paradigms, Wallets, Issuers, and Verifiers. These entities interact over HTTPS using standardised protocols such as OpenID Connect and Security Assertion Markup Language (SAML). Static Software Bill of Materials (SBOMs), derived from source code or build artefacts, detect known vulnerabilities in declared dependencies but cannot identify supply chain attacks where the software is unmodified and a component under the attacker’s control is injected at the configuration or deployment level. This work introduces an approach that derives a runtime SBOM from HTTP messages observed during security testing of IdM deployments, without requiring access to source code, build artefacts, or deployment internals. The approach is implemented within the Micro-Id-Gym Testing Tool, integrated with Burp Suite and Checkmarx ZAP. An evaluation on a proof-of-concept European Digital Identity Wallet implementation shows that the extracted SBOM is identical across proxies and executions, and that HTTP traffic alone is sufficient to identify components with known vulnerabilities, supporting supply chain verification under the Directive on Security of Network and Information Systems (NIS2) and the Cyber Resilience Act (CRA).

Keywords: Security Testing · Software Bill of Materials · Identity Management · Supply Chain · HTTP Proxies

1 Introduction

Identity Management (IdM) deployments are distributed across multiple entities. In federated architectures, they involve Identity Providers (IdPs) and Relying Parties (RPs). In decentralised paradigms based on Self-Sovereign Identity

(SSI) [40], the corresponding roles are filled by Wallets, Issuers, and Verifiers. The interactions among these entities are governed by standardised protocols, primarily OpenID Connect (OIDC) [32], which builds on Open Authorization 2.0 (OAuth) [11], and Security Assertion Markup Language (SAML) [30]. These protocols define how identity claims are packaged, signed, and transmitted across the participating entities. Since they are deployed over HTTPS, an HTTP proxy acting as a Man-in-the-Middle (MITM) [24] is required to inspect the exchanged messages during security testing: the proxy terminates the TLS connection and presents a trusted certificate to the User-Agent, making the underlying HTTP traffic accessible for analysis.

The security of IdM deployments requires several activities, including security testing (see, e.g., [4,5,6]) and the maintenance and analysis of Software Bills of Materials (SBOM) [14,10,9]. Static SBOMs are produced by analysing source code, build manifests, or container images at rest. They are effective for detecting known vulnerabilities in declared dependencies, such as an outdated version of a cryptographic library [31,3].

However, static SBOMs cannot detect a class of supply chain attacks that do not exploit software bugs but instead abuse the protocol logic: the deployment software remains unmodified, while a component under the attacker’s control is injected during configuration or deployment [43,31]. For instance, in an OIDC deployment, a compromised configuration file may redirect the `jwtks_uri` endpoint [20], which exposes the public keys used to verify signed tokens, to a server controlled by the attacker. In this scenario, the software has no Common Vulnerabilities and Exposures (CVEs) and the protocol is used exactly as specified, yet the deployment fetches cryptographic keys from an untrusted source—which can only be detected by observing the actual execution.

A runtime SBOM, derived from the HTTP traffic exchanged during protocol execution, captures the software components and parameters active at runtime rather than those declared in build artefacts. It documents what the deployment *does*, not only what it *is*, and supports supply chain verification under the Directive on Security of Network and Information Systems (NIS2) [14] and the Cyber Resilience Act (CRA) [10]. This is also reflected in joint guidance issued by the U.S. Cybersecurity and Infrastructure Security Agency (CISA), the National Security Agency (NSA), and nineteen international cybersecurity organisations [9].

Runtime SBOMs are a natural by-product of security and conformance testing [7]: while intercepting HTTP messages for testing purposes, component information can be extracted incrementally as more HTTP messages are captured, with little overhead beyond the baseline testing activity. This is particularly relevant in distributed and decentralised deployments, where each entity is operated by a different party. An RP performing security testing interacts with the other entities exclusively through HTTP APIs and has no access to their source code, build artefacts, or static SBOMs. In SSI deployments, the same constraint applies to Wallets, Issuers, and Verifiers. Observed HTTP traffic is the only source

of component information available to the tester without requiring cooperation from the other parties.

Deriving a runtime SBOM from observed HTTP traffic requires proxy-independent observations. Proxies such as Burp Suite (Burp) [38] and Checkmarx ZAP (ZAP) [35] are commonly adopted for security testing of IdM deployments [6,24]. Each proxy records and exposes HTTP messages according to its own internal model. Some proxies deliver complete request and response messages through a single callback; others separate headers and body across multiple asynchronous events and require the consumer to reconstruct the full message. As a result, the same protocol interaction may be represented differently across proxies or across repeated executions of the same proxy. If the same HTTP header carrying a component identifier is visible in one proxy trace but absent from another due to reconstruction ordering, the corresponding component descriptor is produced in one case and not in the other. Any SBOM derived from such observations would depend on the proxy choice rather than on the protocol interaction, making it unsuitable for supply chain verification.

Existing works show that message observations are essential for identifying weaknesses in authentication flows [4,8,29]. Other contributions introduce tools that support automated execution and structured observation of HTTP interactions [5]. These approaches rely on message observations but do not address proxy variability. No existing approach derives an SBOM from observed HTTP traffic, and no existing tool provides a proxy-independent representation of HTTP interactions suitable for this purpose.

This work introduces an approach that derives an SBOM from HTTP message exchanges observed during security testing. The approach relies on a normalisation function that separates proxy-specific behaviour from protocol-visible content and on a pairing process that reconstructs request–response pairs independently of the proxy, constituting the observation layer. Component identification and SBOM generation are handled by a dedicated SBOM pipeline.

The approach is implemented within Micro-Id-Gym Testing Tool (MIG-T) [5], integrated with Burp and ZAP. The approach is validated on a proof-of-concept European Digital Identity (EUDI) Wallet implementation for the POTENTIAL⁴ Large-Scale Pilot, producing a CycloneDX-compliant SBOM [34].

The contributions of this work are summarised as follows:

- A proxy-independent observation model that isolates proxy-specific behaviour and exposes a stable representation of HTTP message exchanges, with stability properties defined to ensure consistent interpretation across proxies and executions.
- An extraction model that derives an SBOM from the normalised observations by identifying software components from HTTP headers, body content, and session identifiers, mapping them to the CycloneDX format to support compliance and supply chain verification under NIS2 and CRA.

⁴ <https://www.digital-identity-wallet.eu/>

- A proxy adapter design that integrates the observation layer without requiring modifications to the proxies, and an SBOM extraction pipeline design that derives component descriptors from normalised HTTP exchanges.
- A full implementation of the approach as the observation layer and the SBOM pipeline within MIG-T, integrated with Burp and ZAP.
- An evaluation on a proof-of-concept EUDI Wallet implementation, showing that normalised representations remain identical across proxies and executions, and that SBOM extraction identifies library and application components with known vulnerabilities from observed HTTP traffic alone.

2 Background

This section introduces the concepts of IdM deployments and their security testing requirements, the SBOM, digital identity wallets, and HTTP proxies.

2.1 Identity Management and Security Testing

IdM protocols define the interactions between entities involved in authentication and authorization flows. Widely adopted protocols include SAML [30] and OIDC [32], which builds on OAuth [11] to delegate authentication and authorization decisions across distributed deployments. These interactions are carried over HTTP and produce message exchanges that can be recorded and analysed during security testing. The correctness of an IdM deployment depends on the content and ordering of these exchanges: deviations from the expected protocol flow may indicate misconfigurations, implementation flaws, or active attacks [4]. The HTTP traffic recorded during the execution of an IdM deployment is an ordered sequence of request–response pairs, where each pair consists of an HTTP request sent by the client and the corresponding HTTP response returned by the server.

MIG-T [5] is a security testing tool for IdM deployments that records HTTP message exchanges through a proxy and analyses them to identify security vulnerabilities, including Cross-Site Request Forgery (CSRF) vulnerabilities [4] and non-compliance with Best Current Practices [8]. It supports Burp and ZAP as proxies.

2.2 Software Bill of Materials

An SBOM is a structured inventory of the software components involved in a deployment, including libraries, frameworks, and runtime dependencies. SBOM formats such as CycloneDX [34] and Software Package Data Exchange (SPDX) [22] provide standardised machine-readable representations for software component inventories [3]. In CycloneDX, each component entry records **type** (library, application, or framework), **name**, **version**, **purl** (a Package URL for automated lookup), **cpe** (for cross-referencing against vulnerability databases such as the

NVD), and **evidence** (an annotation recording the specific artefact or protocol field from which the component was identified).

Existing approaches to SBOM generation differ in the source from which component information is derived [31,18]. Static SBOMs analyse source code, build manifests, or container images [3,48,43], while runtime SBOMs observe the deployment during execution, inspecting loaded libraries or active processes [42].

2.3 Digital Identity Wallets

Digital identity wallets are software components that allow users to store, manage, and present identity credentials in digital form. They act as relying parties in IdM protocol flows, exchanging HTTP messages with identity providers and verifiers to complete authentication and authorization transactions [40]. Wallet implementations are typically deployed on mobile devices and interact with remote backend services through HTTP APIs over HTTPS, producing HTTP message exchanges that are collected through a proxy during security testing.

EUDI Wallet is defined under the revised European Union Digital Identity Regulation (eIDAS 2.0) [15] and provides a standardised framework for digital identity across EU member states. EUDI Wallet implementations rely on OIDC-based protocols for credential issuance and presentation [23,47]. The credential issuance flow follows OpenID for Verifiable Credential Issuance (OpenID4VCI) [23], producing a sequence of HTTP message pairs across multiple endpoints representative of production IdM deployments.

2.4 HTTP Proxies

An HTTP proxy intercepts, records, and optionally modifies the messages exchanged between a client and a server, enabling traffic inspection, manipulation, and replay for security testing purposes. Burp [38] and ZAP [35] are among the most widely adopted proxies for security testing of IdM deployments [6,24]. Each proxy records and exposes HTTP messages according to its own internal model. Burp delivers each complete HTTP transaction in a single callback invocation; ZAP registers separate listeners for requests and responses, each invoked independently. Both proxies expose messages through internal event models that do not guarantee consistent ordering, as headers, bodies, and message updates may be delivered through separate asynchronous callbacks.

Table 1 summarises the main differences between Burp and ZAP. Both proxies provide only partial ordering guarantees, so the sequence in which messages are delivered to registered listeners may differ across executions.

3 Problem Statement and Research Questions

Proxy variability directly affects SBOM quality. Deriving an SBOM from recorded HTTP traffic requires that the same component consistently produces the same observation across proxies and executions. If the same HTTP header is visible in

Table 1: Differences in proxy behaviour.

Aspect	Burp behaviour	ZAP behaviour
Message exposure	Raw request/response	Header/body split
Logging model	Single callback	Multiple listeners
Reconstruction required	No	Yes
Ordering guarantees	Partial	Partial

one proxy trace but absent from another due to reconstruction ordering, the corresponding component descriptor is produced in one case and not in the other. The extracted SBOM then depends on the proxy choice rather than on the protocol interaction. This makes the inventory non-reproducible and unsuitable for supply chain verification under NIS2 and CRA. Stable and proxy-independent observations are therefore a prerequisite for SBOM extraction from HTTP traffic, not an optional property.

No existing approach derives an SBOM from observed HTTP traffic. No existing tool provides a proxy-independent representation of HTTP interactions suitable for this purpose. Addressing this gap requires establishing whether proxy-independent observation is achievable (RQ1), whether the extracted SBOM remains consistent with deployment changes (RQ2), and whether HTTP traffic alone is sufficient to produce a well-formed SBOM (RQ3). The problem leads to the following research questions:

- RQ1** Under what conditions can the representation of an HTTP message exchange be made independent of the proxy used to record it?
- RQ2** Does a change in the observed HTTP traffic, due to a component update or a deployment change, produce a corresponding change in the extracted SBOM?
- RQ3** Can a well-formed SBOM be derived from HTTP traffic alone, without access to the deployment internals, source code, or build artefacts?

Proxy-independence is not an end in itself but the prerequisite for a correct and stable dependency analysis. If the same protocol interaction produces different component descriptors depending on the proxy, the extracted SBOM reflects the proxy choice rather than the deployment. Correctness here means that the SBOM identifies the components actually present in the deployment. Reproducibility means that repeated observations of the same deployment produce the same SBOM. Without proxy-independence, neither property is achievable from HTTP traffic alone.

Each question validates a distinct aspect of the approach. Together they drive the design decisions and the empirical evaluation presented in the following sections.

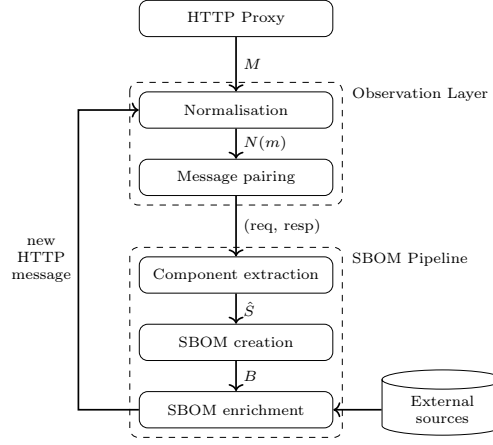


Figure 1: Workflow of the proposed approach.

4 Approach

The proposed approach derives an SBOM from the HTTP messages observed during the execution of an IdM deployment, without requiring access to source code, build artefacts, or deployment internals.

The approach is organised as a pipeline of five stages, illustrated in Figure 1. Raw messages M recorded by the proxy enter the observation layer, where they are first normalised into a proxy-independent representation $N(m)$ and then paired into request–response pairs (req, resp). The resulting pairs are passed to the SBOM pipeline, where component extraction derives a set of component descriptors $\hat{S}$, SBOM creation maps them to a standardised inventory B , and SBOM enrichment extends B with dependency and vulnerability information retrieved from external sources such as the NVD [28], the OpenSSF OSV [33], `deps.dev` [41], and OWASP Dependency-Track [36]. The pipeline is re-executed for each new HTTP message observed by the proxy.

The pipeline processes each (req, resp) pair as it becomes available. When the proxy intercepts a new HTTP message, the observation layer normalises and pairs it, and the SBOM pipeline updates the component inventory accordingly. The resulting SBOM is built incrementally across the captured HTTP messages, without requiring the complete traffic to be available upfront.

4.1 Observation Layer

The observation layer separates what the HTTP protocol defines from what the proxy adds, producing a representation of each message that is independent of the proxy used to record it.

Normalisation. The proxy variability described in Section 2 requires an abstraction layer that separates what the proxy adds from what the protocol defines. Let

M be the set of HTTP messages recorded by the proxy. Each message $m \in M$ may carry proxy-specific metadata such as internal identifiers, parsing offsets, logging timestamps, and reconstruction artefacts introduced during interception. The protocol-visible content of m , denoted $visible(m)$, is the subset of m defined by the HTTP specification [16,17] and observable by any correctly implemented proxy: the request or status line, the header fields, and the body. The observation layer extracts $visible(m)$ and discards proxy-specific components.

The normalisation function $N : M \rightarrow M_{norm}$ maps each recorded message to a canonical representation of $visible(m)$, retaining only protocol-defined fields as specified in [16,17] and applying a fixed serialisation order. The distinction between $visible(m)$ and $N(m)$ is that $visible(m)$ identifies the protocol-defined content of m , while $N(m)$ is its canonical serialised form used for comparison and processing. For any two recorded messages m_1 and m_2 such that $visible(m_1) = visible(m_2)$, normalisation preserves their equality: $N(m_1) = N(m_2)$, thus addressing RQ1: proxy-independent observation is achievable by normalising to protocol-visible content.

Message pairing. Request–response pairs are reconstructed using only normalised messages. Each request m_{req} is matched to its response m_{res} using the protocol-visible identifiers in $N(m_{req})$, specifically the request URI and method; the resulting pair is $G = (N(m_{req}), N(m_{res}))$. The sequence of pairs is ordered by the position of each request in the observed trace: requests are sorted by their order of appearance in the protocol-visible content, so that two requests to the same URI are distinguished by their relative position. Because the ordering criterion depends exclusively on protocol-visible content, proxy-level buffering, event delivery order, and metadata do not affect the resulting sequence. A deployment change that modifies a response header—for instance, a library update that changes the value of the **Server** field—produces a different $N(m_{res})$ and therefore a different component descriptor in the extracted SBOM, thus addressing RQ2: any change observable at the protocol level is reflected in the extracted SBOM. Two executions of the same interaction recorded by different proxies produce the same sequence of pairs when the protocol-visible content is identical.

4.2 SBOM Pipeline

The SBOM pipeline operates on the request–response pairs produced by the observation layer and derives a runtime SBOM from their protocol-visible content.

Component extraction. Each normalised message is inspected for protocol-visible fields carrying component information. HTTP messages [16,17] are structured into a request or status line, a set of header fields, and an optional body. Component information may appear in each part: request headers expose the client stack (e.g., **User-Agent**: curl/7.88.1), response headers expose the server stack (e.g., **Server**: Apache/2.4.57, **X-Powered-By**: Express), versioned paths and meta tags in the response body expose Content Management System (CMS) versions (e.g., `<meta name="generator" content="WordPress 6.1">`), and session cookie names expose the session management framework (e.g., **Set-Cookie**:

`JSESSIONID=ABC123; Path=/; HttpOnly` identifies a Java Servlet container such as Tomcat). The identified fields are passed to the SBOM creation stage.

SBOM Creation. The resulting SBOM is a runtime SBOM: it reflects the components active and observable during protocol execution, not those declared in build artefacts or source manifests. Three structural limitations apply. First, components that produce no observable network traffic are not captured. Second, components that suppress version information are recorded without a version string. Third, the dependency tree of an identified component is not observable from HTTP traffic: headers disclose top-level identifiers but carry no information about transitive dependencies. These limitations are inherent to black-box observation and are recorded in the generated SBOM through the **evidence** annotation.

The approach detects the supply chain attack described in Section 1 at the enrichment step. When the client fetches public keys from the redirected `jwtks_uri` endpoint, the **Server** response header identifies the component serving the request. If that component does not belong to the expected deployment, the discrepancy appears in the runtime SBOM as an unrecognised entry. Moreover, the approach assumes that components report their identity truthfully in HTTP headers. A compromised component that sends a fake **Server** or **User-Agent** value would be recorded with the declared identity rather than the real one, a limitation shared by all passive traffic observation approaches.

Let C be the set of all possible component descriptors. Each descriptor $c \in C$ is a tuple $c = (name, version, type, sources)$, where $sources = [s_1, \dots, s_k]$ is the list of HTTP fields from which the component was identified (e.g., **Server**, **User-Agent**), not the entities in the deployment (such as IdP, UA, or RP) that generated the messages. An extraction function $E : M_{norm} \rightarrow \mathcal{P}(C)$ maps each normalised message to a set of descriptors, since a single message may carry information about more than one component. S is the union of all descriptors from every normalised pair, so no observable component is omitted. The complete set S is:

$$S = \bigcup_{i=1}^n (E(N(m_{req}^i)) \cup E(N(m_{res}^i)))$$

When the same component is observed in more than one HTTP field, S contains multiple entries with the same *name* and *version* but distinct *sources*. Deduplication consolidates these into a single descriptor carrying the full provenance record $[s_1, \dots, s_k]$, producing $\hat{S}$. Each entry in $\hat{S}$ is then mapped to a component record in the SBOM document B in CycloneDX format, with its *sources* list recorded as an **evidence** annotation. Thus, S is the raw extracted set, $\hat{S}$ its deduplicated form, and B the CycloneDX document derived from $\hat{S}$.

Every component observable through the recorded HTTP traffic produces a descriptor in $\hat{S}$ and a corresponding entry in B —no observable component is omitted, thus addressing RQ3: a well-formed SBOM can be derived from HTTP traffic alone. Since E operates exclusively on normalised messages, the extracted SBOM inherits the stability properties of the observation layer: for two

executions of the same interaction recorded by different proxies, the extracted component sets are identical when the protocol-visible content is identical.

Following the classification of SBOM information sources in [3], the protocol-visible fields that carry component information are organised into four categories: request headers, response headers, response body, and cookie names. Each category exposes information about a specific layer of the software stack and is associated with known attack techniques that exploit the disclosed identifiers to target vulnerable components or hijack sessions. Table 2 summarises the representative HTTP fields, component types, and known attacks for each category.

SBOM Enrichment. The SBOM document B produced by the creation stage captures the components observable from HTTP traffic but is bounded by what the deployment discloses. In particular, headers identify top-level library identifiers but carry no information about transitive dependencies or known vulnerabilities. An enrichment stage extends B by querying external sources using the identifiers recorded in each component entry.

A **purl** (Package URL) is a standardised, machine-readable identifier that encodes the package ecosystem, name, and version in a uniform format (e.g., `pkg:maven/commons-io/commons-io@2.11.0`) [13]. For components belonging to structured ecosystems, the **purl** provides sufficient information to query the corresponding package registry and retrieve the declared dependencies, allowing the transitive dependency tree to be reconstructed. For components assigned a generic **purl** type—used when the package does not belong to a structured ecosystem such as Maven, npm, PyPI, or NuGet—dependency resolution is not supported, as no registry provides a machine-readable dependency manifest for such identifiers.

Known vulnerabilities associated with identified components are retrieved by querying public vulnerability databases using the **purl** or **cpe** recorded in each component entry. This can be achieved either by querying sources such as the NVD [28] and the OpenSSF OSV [33] directly, or by delegating vulnerability discovery to component analysis platforms such as OWASP Dependency-Track [36], which accept an SBOM as input and aggregate multiple vulnerability databases for CVE matching [10].

5 Implementation

The observation layer is implemented as a proxy adapter integrated into MIG-T, with dedicated adapters for Burp⁵ and ZAP.⁶ The implementation buffers partial data and reconstructs complete request-response pairs, serialised into a textual representation where each pair is separated by a delimiter (#). Only HTTP messages generated by MIG-T during test execution are retained.

⁵ <https://portswigger.net/burp/extend/api/burp/ihttplistener.html>

⁶ <https://javadoc.io/static/org.zaproxy/zap/2.8.0/org/zaproxy/zap/network/HttpSenderListener.html>

Table 2: Categories of HTTP fields used for component identification.

Category	Representative fields	Component type	Known attacks
Request headers	User-Agent	Client library, browser, mobile framework	Client fingerprinting, version-targeted exploits [49]
Response headers	Server, X-Powered-By, X-AspNet-Version, Via, X-Cache	Server, framework, proxy, cache	Banner grabbing, CVE-targeted exploitation [28,12]
Response body	<meta generator>, versioned asset paths	CMS, build toolchain	CMS version inference, path disclosure [12]
Cookie names	JSESSIONID, PHPSESSID, ASP.NET_SessionId	Session framework	Session fixation, framework fingerprinting [44,21]

The SBOM pipeline is implemented as a Python script that operates on the normalised request–response pairs produced by the observation layer. The script applies the extraction function E to each normalised message and aggregates the resulting descriptors into a CycloneDX 1.6-compliant [34] SBOM document in JSON format. During enrichment, each component is associated with a `purl` retrieved from public registries (e.g., Maven, `deps.dev`), or assigned a `pkg:generic` fallback when no match is found, and a CPE generated systematically for each component. Compatibility has been validated with Dependency-Track [36] and Trivy [1]. Experiments were conducted on a MacBook Air (Apple M1, 16 GB RAM, macOS). The full implementation is available at the companion website of this paper [45].

6 Case Study

The credential issuance flow is the protocol sequence through which a Wallet Instance obtains a verifiable credential from a Credential Issuer. It follows the OpenID4VCI specification [23] and involves the Wallet Instance, a Wallet Provider Backend that registers and attests the wallet, and a Credential Issuer that issues the credential after authentication. The flow produces a sequence of HTTP request–response pairs across multiple endpoints, representative of the complexity found in production IdM deployments.

To evaluate the proposed approach on a representative IdM deployment, the credential issuance flow of the `io-eudiw-app` application is used as the case study. The application is developed by PagoPA S.p.A., a proof-of-concept open-source EUDI Wallet implementation for Android, deployed within the POTENTIAL Large-Scale Pilot [37]. Two versions are evaluated: v2.1.1 and v2.6, to assess whether the approach correctly reflects component changes across releases. The application is executed in a controlled environment where MIG-T replays user interactions to trigger the credential issuance workflow.

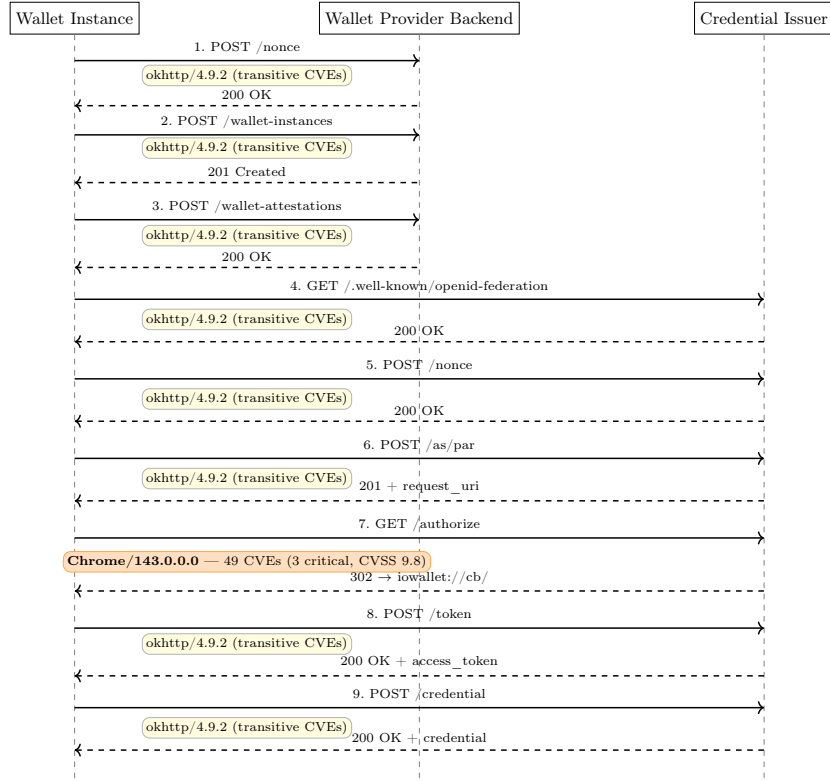


Figure 2: Simplified MSC of the credential issuance flow with component and vulnerability annotations.

During execution, HTTP traffic generated by the application is intercepted through both Burp and ZAP via the observation layer. Ten executions are performed per proxy to verify the stability of the normalised output across repeated runs. The intercepted HTTP messages are normalised according to the approach described in Section 4, ensuring that equivalent request–response pairs are obtained independently of the underlying proxy. The extracted SBOM is identical regardless of the proxy used to record the traffic. The runtime SBOM is built incrementally: each execution contributes additional observations, making it a natural by-product of the security testing activity.

Figure 2 presents a simplified Message Sequence Chart (MSC) [19] of the credential issuance flow for version 2.1.1, with identified components annotated on the corresponding requests; it reflects the HTTP message exchanges observed during proxy interception and does not reproduce the full internal logic of the wallet initialisation flow as defined in the specification. The execution involves interactions between the mobile application (Wallet Instance) and two backend services deployed as Azure Function apps [25] (Wallet Provider Back-

end and Credential Issuer), implementing the registration and credential issuer roles, as reported in Figure 2. Each execution generates 9 request–response pairs across 8 distinct endpoint paths, including `/nonce`, `/wallet-instances`, `/wallet-attestations`, `/.well-known/openid-federation`, `/as/par`, `/authorize`, `/token`, and `/credential`. The `/nonce` endpoint is called on both the Wallet Provider Backend and the Credential Issuer, producing two distinct pairs on the same path but on different hosts, which accounts for the difference between the number of pairs and the number of distinct paths.

The SBOM pipeline is executed on the normalised message pairs, performing component extraction, SBOM creation, and SBOM enrichment, and producing a CycloneDX 1.6 document as output. Two components are identified from the `User-Agent` headers of the Wallet Instance. The first is `okhttp/4.9.2`, the HTTP client library used by the application for backend communication. The second is `Google Chrome for Android` version 143.0.0.0, corresponding to the browser engine invoked during the authorization. Components from the Wallet Provider Backend and Credential Issuer (e.g., Kestrel and the Azure Functions runtime) are present in response headers but do not expose version information. To avoid introducing uncertainty, these components are excluded from the SBOM.

The generated SBOM is then used as input to vulnerability analysis tools, including [36,1], which correlate the identified components with known vulnerabilities from public databases [28]. The analysis reports 49 known vulnerabilities associated with `Google Chrome for Android` version 143.0, including 3 critical vulnerabilities (CVSS 9.8) related to memory corruption and remote code execution. Among the high-severity vulnerabilities, representative examples include CVE-2026-0906, CVE-2026-3932, CVE-2026-3936, CVE-2026-4439. No vulnerabilities are directly associated with `okhttp/4.9.2`, though transitive ones may be present. As shown in Figure 2, the component highlighted in orange on message 7 is directly associated with known CVEs retrieved from the NVD, while those highlighted in yellow on the remaining messages (1–6, 8–9) carry transitive CVEs. Listings 3a and 3b report the generated component descriptors, including their associated vulnerability data.

The comparison between the SBOMs extracted from the two tested versions illustrates how the approach supports supply chain verification in practice. In version 2.1.1, the extracted SBOM identifies `okhttp/4.9.2`; in version 2.6, the same extraction identifies `okhttp/4.12.0`, which carries no known vulnerabilities. This change is detectable from HTTP traffic alone, without access to the source code or build artefacts.

The scenario is representative of a supply chain substitution: if a build tool were compromised to inject a vulnerable version of `okhttp` in place of the declared one, the discrepancy would be visible as a mismatch between the component identifier recorded in the static SBOM and the one extracted from the observed traffic. Static analysis of `build.gradle` would report the declared dependency; the runtime SBOM would report the component actually loaded.

```

"components": [
  {
    "bom-ref": "1-okhttp",
    "type": "library",
    "name": "okhttp",
    "version": "4.9.2",
    "purl": "pkg:maven/com.squareup.okhttp3/okhttp@4.9.2",
    ...
  },
  {
    "bom-ref": "2-okio",
    "type": "library",
    "name": "okio",
    "version": "2.8.0",
    "purl": "pkg:maven/com.squareup.okio/okio@2.8.0",
    ...
  }, ...
], ...
"dependencies": [
  {
    "ref": "1-okhttp",
    "dependsOn": [
      "2-okio", ...
    ]
  }, ...
],
"vulnerabilities": [
  {
    "id": "CVE-2023-3635",
    "ratings": [
      {
        "source": {
          "name": "nvd"
        },
        "score": 7.5,
        "severity": "high",
        "method": "CVSSv31",
        "vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H"
      }, ...
    ], ...
  }, ...
]
"affects": [
  {
    "ref": "2-okio",
    "versions": [
      {
        "version": "2.8.0",
        "status": "affected"
      }, ], ], ], ]

```

(a) okhttp 4.9.2.

```
{
  "components": [
    {
      "type": "application",
      "bom-ref": "417e7a95-1e46-45aa-9a74-4ad5c3384505",
      "name": "Chrome Mobile",
      "version": "143.0.0",
      "cpe": "cpe:2.3:a:google:chrome:143.0.0:*:*:*:*:*:*",
      "purl": "pkg:generic/google-chrome@143.0.0"
    },
    ...
  ],
  ...
},
{
  "vulnerabilities": [
    {
      "bom-ref": "c2738ca9-5c58-4d6b-9fb1-ebf7d957c2c5",
      "id": "CVE-2026-0906",
      "source": {
        "name": "NVD",
        "url": "https://nvd.nist.gov/"
      },
      "ratings": [
        {
          "source": {
            "name": "NVD",
            "url": "https://nvd.nist.gov/"
          },
          "score": 9.8,
          "severity": "critical",
          "method": "CVSSv31",
          "vector": "CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H"
        }
      ],
      "cves": [
        451
      ],
      "description": "Incorrect security UI in Google Chrome on Android prior to 144.0.7559.59 allowed a remote attacker to spoof the contents of the Omnibox (URL bar) via a crafted HTML page. ( Chromium security severity: Low)",
      "published": "2026-01-20T05:16:16Z",
      "updated": "2026-01-29T20:27:23Z",
      "affects": [
        {
          "ref": "417e7a95-1e46-45aa-9a74-4ad5c3384505"
        }
      ],
      },
    ],
  ],
}
```

(b) Google Chrome for Android 143.

Figure 3: Excerpts of the generated CycloneDX descriptors.

Comparing the two reveals the substitution with no access to the deployment internals and no overhead beyond the testing activity already performed.

To further validate the approach, the experiment was repeated using an alternative browser engine, Firefox for Android 148.0. The analysis correctly identified known vulnerabilities associated with this version, including CVE-2026-4716, CVE-2026-4723, and CVE-2026-4702, all affecting Firefox prior to version 149, as confirmed against the NVD [28] and Mozilla security advisories [27]. Two versions of the EUDI Wallet application (v2.1.1 and v2.6) were also evaluated, showing consistent component identification across both; in v2.6, the `okhttp` dependency was updated to version 4.12.0, which carries no known vulnerabilities, confirming that the approach correctly reflects the underlying software updates. To corroborate the results, the APK binary of the EUDI Wallet application was analysed using blint [46], a static binary analysis tool for SBOM generation

Table 3: Message exposure structures across proxies.

Proxy	Message structure	Async events	Supports $N(m)$
Burp [38]	Raw message	Yes	Yes
ZAP [35]	Header/body split	Yes	Yes
mitmproxy [26]	Header/body split	Yes	Yes
Charles Proxy [39]	Raw message	No	Yes

from binary executables. Due to bytecode obfuscation and class merging applied during the release build, neither `okhttp` nor its version string were recoverable from the binary, confirming that HTTP traffic observation provides component visibility that static binary analysis cannot achieve.

All results, including identified components, versions, dependencies, and associated vulnerabilities, were manually verified and reported on the companion website [45].

7 Related Work

The approach introduced in this work sits at the intersection of HTTP traffic observation through proxies and SBOM generation from software deployments.

7.1 HTTP Proxies and Message Observation

HTTP proxies differ substantially in how they expose intercepted messages to external tools. Burp [38] and Charles Proxy [39] deliver complete request–response pairs through a single callback, whereas ZAP [35] and mitmproxy [26] separate header and body delivery across asynchronous listeners.

No existing work proposes a proxy-independent normalisation layer that abstracts over these structural differences. As shown in Table 3, all proxies examined support $N(m)$, confirming that the observation layer is not tied to a specific proxy architecture.

7.2 SBOM Generation

SBOM generation has received increasing attention driven by regulatory requirements such as NIS2 and CRA [14,10]. Static analysis tools extract component information from package manifests, lock files, and binary metadata [48,2], with considerable divergence in the components identified [31,18]. Source-code-based approaches focus on components referenced in the codebase [3]. Runtime approaches monitor library calls during execution [42], requiring white-box access to the running process. The approach instead observes HTTP traffic passively, without access to the deployment internals. Table 4 summarises the main characteristics of existing approaches. None operates without access to source code, build scripts, or deployment internals, and none derives an SBOM from observed

Table 4: Comparison of SBOM generation approaches.

Work	Source	Type	White-box	HTTP traffic
Xia et al. [48]	Source code	Static	Yes	No
Balliu et al. [2]	Binary artefacts	Static	Yes	No
Bangash et al. [3]	Source code	Static	Yes	No
O'Donoghue et al. [31]	Build manifest	Static	Yes	No
Halbritter et al. [18]	Container image	Static	Yes	No
Sharma et al. [42]	Runtime process	Runtime	Yes	No
<i>Our Approach</i>	HTTP traffic	Runtime	No	Yes

HTTP traffic. This work fills this gap by producing a component inventory exclusively from the protocol-visible content of intercepted HTTP messages, inheriting the stability properties of the normalised observation layer.

7.3 Supply Chain Attack Detection

Supply chain attacks on software deployments have received increasing attention in recent years [14,10]. Existing detection techniques operate at the build and distribution level: they verify the integrity of build artefacts, check package signatures, and monitor dependency resolution [3,48]. They cannot detect attacks that operate at the configuration or deployment level, where the software artefact is unmodified but the runtime behaviour is altered by substituting a configuration parameter or redirecting a service endpoint. This gap motivates the work presented in this paper.

8 Conclusion and Future Work

This work introduced an approach for extracting a runtime SBOM from HTTP messages observed during security testing of IdM deployments. The normalisation function and pairing mechanism constitute the observation layer, which reconstructs request–response pairs independently of the proxy and maps them to a CycloneDX-compliant inventory through the SBOM pipeline. Both are implemented within MIG-T, integrated with Burp and ZAP. The case study on the credential issuance flow of a proof-of-concept EUDI Wallet implementation shows that normalised observations are identical across proxies and executions, and that the extracted SBOM reflects component changes across application versions. The integration with Dependency-Track confirms that an SBOM derived from intercepted HTTP traffic can support automated vulnerability assessment. The analysis identified components with known vulnerabilities, including critical-severity entries, demonstrating that a partial black-box inventory can support supply chain verification under NIS2 and CRA without access to source code or build artefacts.

Future work includes extending the observation layer to other HTTP-based deployments, improving transitive dependency coverage through typical dependency tree libraries, and integration into Continuous Delivery pipelines [7].

References

1. Aqua Security: Trivy (2024), <https://trivy.dev/>, accessed: 2026-04-09
2. Balliu, M., Baudry, B., Bobadilla, S., Ekstedt, M., Monperrus, M., Ron, J., Sharma, A., Skoglund, G., Soto-Valero, C., Wittlinger, M.: Challenges of Producing Software Bill of Materials for Java. *IEEE Security & Privacy* **21**(6), 12–23 (2023)
3. Bangash, A.A., Ge, T., Zhao, Z., Singh, A., Wang, Z., Adams, B.: The State of the SBOM Tool Ecosystems: A Comparative Analysis of SPDX and CycloneDX. *arXiv preprint arXiv:2512.21781* (2025)
4. Bisegna, A., Bitussi, M., Carbone, R., Compagna, L., Ranise, S., Sudhodanan, A.: CSRFing the SSO Waves: Security Testing of SSO-Based Account Linking Process. In: *2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P)*. pp. 139–154. IEEE (2024)
5. Bisegna, A., Bitussi, M., Carbone, R., Ranise, S.: Enhancing Security Testing for Identity Management Implementations: Introducing Micro-Id-Gym Language and Micro-Id-Gym Testing Tool. *IEEE Security & Privacy* (2024)
6. Bisegna, A., Carbone, R., Pellizzari, G., Ranise, S.: Micro-Id-Gym: A Flexible Tool for Pentesting Identity Management Protocols in the Wild and in the Laboratory. In: *International Workshop on Emerging Technologies for Authorization and Authentication*. pp. 71–89. Springer (2020)
7. Bisegna, A., Carbone, R., Cristiano, L., De Matteis, P., Ranise, S.: Towards Continuous Risk Assessment and Conformance Checking of IdM Deployments. In: *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. pp. 555–560. IEEE (2025)
8. Bisegna, A., Carbone, R., Ranise, S.: Integrating a Pentesting Tool for IdM Protocols in a Continuous Delivery Pipeline. In: *International Workshop on Emerging Technologies for Authorization and Authentication*. pp. 94–110. Springer (2021)
9. CISA, NSA: A Shared Vision of Software Bill of Materials (SBOM) for Cybersecurity. Tech. rep., U.S. Cybersecurity and Infrastructure Security Agency (Sep 2025), <https://www.cisa.gov/resources-tools/resources/shared-vision-software-bill-materials-sbom-cybersecurity>
10. Corti, G., Sasseti, G., Sharif, A., Ponta, S.E., Rizzi, M., De Matteis, P., Piras, L., Carbone, R., Ranise, S.: A First Appraisal of NIS2 and CRA Compliance Leveraging Open Source Tools. In: *2025 IEEE 33rd International Requirements Engineering Conference Workshops (REW)*. pp. 269–274. IEEE (2025)
11. D. Hardt: The OAuth 2.0 Authorization Framework. <https://datatracker.ietf.org/doc/html/rfc6749> (2012)
12. Darwinkel, P.: Fingerprinting web servers through Transformer-encoded HTTP response headers (2024), <https://arxiv.org/abs/2404.00056>
13. Ecma International: Package URL (PURL) Specification. Tech. Rep. ECMA-427, Ecma International (2024), <https://ecma-international.org/publications-and-standards/standards/ecma-427/>
14. European Parliament and Council of the European Union: Directive (EU) 2022/2555 of the European Parliament and of the Council of 14 December 2022 on Measures for a High Common Level of Cybersecurity Across the Union (NIS2 Directive). <https://eur-lex.europa.eu/eli/dir/2022/2555> (2022), accessed: 2026-04-09
15. European Parliament and Council of the European Union: Regulation (EU) 2024/1183 of the European Parliament and of the Council of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European

- Digital Identity Framework. Regulation 2024/1183, Official Journal of the European Union (2024), <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1183>, entered into force 20 May 2024
16. Fielding, R., Reschke, J.: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing. RFC 7230 (2014), <https://datatracker.ietf.org/doc/html/rfc7230>
 17. Fielding, R., Reschke, J.: Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content. RFC 7231 (2014), <https://datatracker.ietf.org/doc/html/rfc7231>
 18. Halbritter, A., Merli, D.: Accuracy Evaluation of SBOM Tools for Web Applications and System-Level Software. In: Proceedings of the 19th International Conference on Availability, Reliability and Security (ARES). ACM (2024)
 19. ITU-T: Z.120: Message Sequence Chart (MSC). ITU-T Recommendation Z.120, International Telecommunication Union (2011), <https://www.itu.int/rec/T-REC-Z.120>
 20. Jones, M.: JSON Web Key (JWK). <https://www.rfc-editor.org/rfc/rfc7517> (May 2015). <https://doi.org/10.17487/RFC7517>
 21. Kolsek, M.: Session Fixation Vulnerability in Web-based Applications. Tech. rep., Acros Security (2002), https://acrosssecurity.com/papers/session_fixation.pdf
 22. Linux Foundation: SPDX Specification V3.0.1 – Software Package Data Exchange. ISO/IEC Standard ISO/IEC 5962:2021, Linux Foundation (2024), <https://spdx.github.io/spdx-spec/v3.0.1/>
 23. Lodderstedt, T. and Yasuda, K. and Looker, T. and Bastian, P.: OpenID for Verifiable Credential Issuance 1.0. OpenID Foundation (2025), https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html
 24. Mainka, C., Somorovsky, J., Schwenk, J.: Penetration Testing Tool for Web Services Security. In: 2012 IEEE Eighth World Congress on Services. pp. 163–170. IEEE (2012)
 25. Microsoft: Azure Functions Overview. <https://learn.microsoft.com/en-us/azure/azure-functions/functions-overview>, accessed: 2026-04-09
 26. mitmproxy Project: mitmproxy Documentation. <https://docs.mitmproxy.org/>, accessed: 2026-04-09
 27. Mozilla: Mozilla Foundation Security Advisory 2026-20. <https://www.mozilla.org/en-US/security/advisories/mfsa2026-20/> (2026), accessed: 2026-04-09
 28. National Institute of Standards and Technology: National Vulnerability Database (NVD) (2024), <https://nvd.nist.gov/>
 29. Nóbrega Gonçalves, S.M., Tomasi, A., Bisegna, A., Pellizzari, G., Ranise, S.: Verifiable Contracting: A Use Case for Onboarding and Contract Offering in Financial Services with eIDAS and Verifiable Credentials. In: European Symposium on Research in Computer Security. pp. 133–144. Springer (2020)
 30. OASIS Security Services Technical Committee: SAML V2.0 Technical Overview. <https://docs.oasis-open.org/security/saml/Post2.0/sstc-saml-tech-overview-2.0.html> (2008)
 31. O’Donoghue, E., Boles, B., Izurieta, C., Reinhold, A.: Impacts of Software Bill of Materials (SBOM) Generation on Vulnerability Detection. In: Proceedings of the 2024 Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED). ACM (2024)
 32. OpenID Foundation: OpenID Connect Core 1.0. https://openid.net/specs/openid-connect-core-1_0.html, accessed: 2026-04-09
 33. OpenSSF: OSV: A distributed vulnerability database for open source (2024), <https://osv.dev/>

34. OWASP: CycloneDX Specification. <https://cyclonedx.org/specification/overview/>, accessed: 2026-04-09
35. OWASP Foundation: Checkmarx ZAP Documentation. <https://www.zaproxy.org/docs/>, accessed: 2026-04-09
36. OWASP Foundation: Dependency-Track. <https://dependencytrack.org>, accessed: 2026-04-09
37. PagoPA S.p.A.: io-eudiw-app, version 2.1.1. <https://github.com/pagopa/io-eudiw-app/releases/tag/2.1.1> (2025), accessed: 2026-04-09
38. PortSwigger Ltd.: Burp Suite Documentation. <https://portswigger.net/burp/documentation>, accessed: 2026-04-09
39. von Randow, K.: Charles Proxy. <https://www.charlesproxy.com/>, accessed: 2026-04-09
40. Salah, M.B.H., Laborde, R., Canavese, D., Benzekri, A., Kandi, M., Ferreira, A.: XCIId: An SSI-Based Cross-Cloud Identity Wallet. In: IEEE Conference on Communications and Network Security (IEEE CNS 2025). Avignon, France (2025)
41. Sarnesjo, J., Ringland, N.: Announcing the deps.dev API: Critical Dependency Data for Secure Supply Chains. Google Online Security Blog, <https://security.googleblog.com/2023/04/announcing-depsdev-api-critical.html> (2023), accessed: 2026-04-09
42. Sharma, A., Wittlinger, M., Baudry, B., Monperrus, M.: SBOM.EXE: Countering Dynamic Code Injection based on Software Bill of Materials in Java. arXiv preprint arXiv:2407.00246 (2024)
43. Sorocean, O., Portillo-Dominguez, A., Ayala-Rivera, V.: Securing the Software Supply Chain with Software Bill of Materials (SBOMs): An Empirical Evaluation of Open-Source Tools in Enterprise IT Environments. vol. 51, pp. 824–847. Springer (2025)
44. Squarcina, M., Adão, P., Veronese, L., Maffei, M.: Cookie Crumbles: Breaking and Fixing Web Session Integrity. In: Proceedings of the 32nd USENIX Security Symposium. USENIX (2023)
45. Companion Website. <https://sites.google.com/fbk.eu/dbsec2026> (2026), accessed: 2026-04-09
46. Team AppThreat - OWASP Project: blint: Binary Linter and SBOM Generator. <https://github.com/owasp-dep-scan/blint> (2025), accessed: 2026-04-09
47. Terbu, O., Lodderstedt, T., Yasuda, K., Looker, T.: OpenID for Verifiable Presentations 1.0. https://openid.net/specs/openid-4-verifiable-presentations-1_0.html (2024)
48. Xia, B., Bi, T., Xing, Z., Lu, Q., Zhu, L.: An Empirical Study on Software Bill of Materials: Where We Stand and the Road Ahead. In: Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE). pp. 2630–2642. IEEE (2023)
49. Zhang, Y., Mekky, H., Zhang, Z.L., Torres, R., Lee, S.J., Tongaonkar, A., Mellia, M.: Detecting Malicious Activities with User-Agent Based Profiles. International Journal of Network Management (2015)